# Infinite-ISP Reference Model v1.01

## Description of Algorithms

## 23rd October 2026

# About this Document:

## Purpose

This document is a supplement provided with the Infinite-ISP Reference Model, it provides algorithm details of all modules in the ISP as well as the explanation of configuration parameters.

## Revision History

| Version | Released Date | Change Description |
|---|---|---|
| **1.00** | 2023-06-01 | Initial Draft – Reference Model V1 |
| **1.01** | 2023-09-09 | Added Sharpening Module |
| **1.01** | 2026-10-23 | Added Local dynamic contrast improvement Module |

# Contents

Contents ..... 3
List of Figures ..... 6
List of Tables ..... 7
Introduction ..... 8
Motivation ..... 9
Modules Description ..... 10
Crop ..... 11
Dead Pixel Correction ..... 12
Algorithm Explanation ..... 12
Configuration Parameters ..... 13
Black Level Correction ..... 14
Algorithm Explanation ..... 14
Configuration Parameters ..... 15
Opto-Electronic Conversion Function—OECF ..... 16
Algorithm Explanation ..... 16
Configuration Parameters ..... 16
Bayer Noise Reduction ..... 17
Algorithm Explanation ..... 17
Configuration Parameters ..... 18
Digital Gain ..... 19
Algorithm Explanation ..... 19
Configuration Parameters ..... 19
White Balance ..... 21
Algorithm Explanation: ..... 21
Configuration Parameters ..... 21
3A STATS ..... 22
Auto White Balance ..... 23
Algorithm Explanation ..... 23
Configuration Parameters ..... 24

Auto Exposure ..... 25
Algorithm Explanation ..... 25
Configuration Parameters ..... 26
Color Filter Array ..... 27
Algorithm Explanation ..... 27
Configuration Parameters ..... 33
Color Correction Matrix ..... 34
Algorithm Explanation ..... 34
Configuration Parameters ..... 34
Gamma Correction ..... 35
Algorithm Explanation ..... 35
Configuration Parameters ..... 35
Color Space Conversion ..... 36
Algorithm Explanation ..... 36
Configuration Parameters ..... 37
Local Dynamic Contrast Improvement ..... 38
Algorithm Explanation ..... 38
Configuration Parameters ..... 39
Sharpening ..... 40
Algorithm Explanation ..... 40
Configuration Parameters ..... 40
2D Noise Reduction ..... 41
Algorithm Explanation ..... 41
Configuration Parameters ..... 42
RGB Conversion ..... 43
Configuration Parameters ..... 44
Invalid Region Crop ..... 45
Configuration Parameters ..... 45
Scale ..... 46
Algorithm Explanation ..... 46
Configuration Parameters ..... 47

YUV Format – 444-422 ..... 48
Algorithm Explanation ..... 48
Configuration Parameters ..... 50
Pipeline Results ..... 51
IQ Metrics Analysis ..... 53
References ..... 54

# List of Figures

Figure 1 : Infinite-ISP Reference Model pipeline ..... 8
Figure 2 : Defective and corrected pixels ..... 12
Figure 3 : BNR algorithm ..... 17
Figure 4 : Flowchart for the Malwar He Cutler's Demosaicing Algorithm ..... 28
Figure 5 : Filter Coefficients for linear interpolation of R, G & B data. ..... 29
Figure 6 : Raw Image with RGGB Bayer pattern ..... 29
Figure 7 : Masking channels ..... 30
Figure 8 : Convolution of a raw image with Filter Type 1 ..... 30
Figure 9 : Estimation of g at r and b locations ..... 30
Figure 10 : Final G channel ..... 31
Figure 11 : Location of R pixels masks ..... 31
Figure 12 : Convolution of Type 2 filters with raw image for R estimated images ..... 32
Figure 13 : Extracting estimated R values ..... 32
Figure 14 : Final R Channel ..... 33
Figure 15 : LDCI Tile boundary blending ..... 39
Figure 16 : Pictorial representation of 4:4:4 and 4:2:2 ..... 48
Figure 17 : Memory view of bytes for packed format ..... 49
Figure 18 : 4:4:4 format ..... 49
Figure 19 : 4:2:2 format ..... 49
Figure 20 : Memory view of 4:2:2 ..... 50
Figure 21 : Subsample entries for 4:2:2 ..... 50
Figure 22 : Pipeline Results ..... 52

# List of Tables

Table 1 : Crop configuration parameters ..... 11
Table 2 : DPC configuration parameters ..... 13
Table 3 : BLC configuration parameters ..... 15
Table 4 : OECF configuration parameters ..... 16
Table 5 : BNR configuration parameters ..... 18
Table 6 : DG configuration parameters ..... 20
Table 7 : WB configuration parameters ..... 21
Table 8 : AWB configuration parameters ..... 24
Table 9 : AE configuration parameters ..... 26
Table 10 : CCM configuration parameters ..... 34
Table 11 : GC configuration parameters ..... 35
Table 12 : CSC configuration parameters ..... 38
Table 13 :LDCI configuration parameters ..... 39
Table 14 : Sharpening Configuration Parameters ..... 40
Table 15 : 2DNR configuration parameters ..... 42
Table 16 : RGBC configuration parameters ..... 44
Table 17 : IRC configuration parameters ..... 45
Table 18 : Scaling – Valid Output Sizes ..... 47
Table 19 : Scale configuration parameters ..... 47
Table 20 : YUV Formats configuration parameters ..... 50
Table 21 : IQ Metrics Analysis ..... 53

# Introduction

Infinite-ISP Reference Model (aka Model) is a collection of 20 Python modules which convert an input RAW image from a sensor to an output YUV image. All the modules are individually configurable via various parameters. Some of the module's configurable parameters are determined by using a separate software tuning tool while others are more directly determined. The model pipeline is shown in Figure 1.

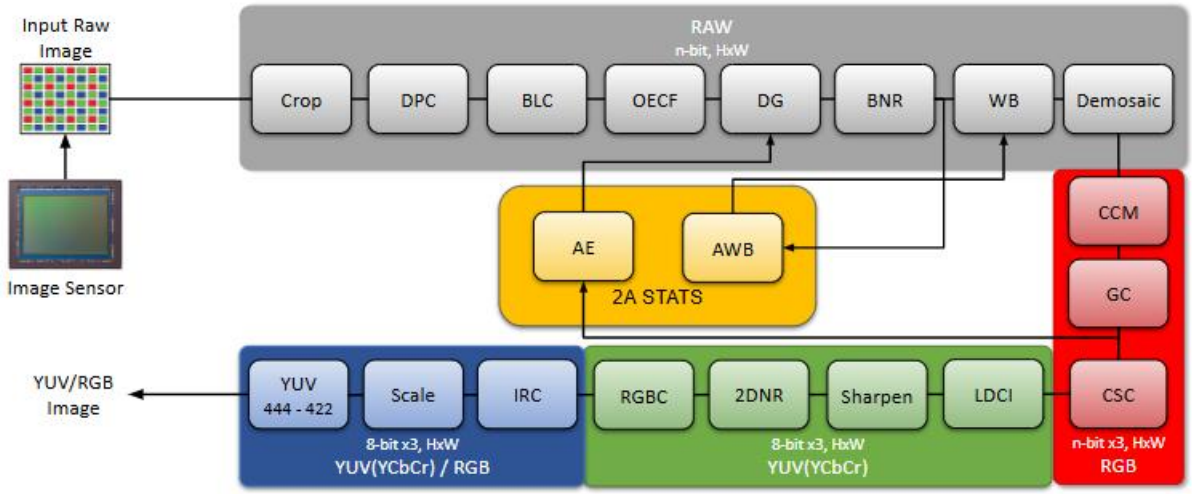



Figure 1: Infinite-ISP Reference Model pipeline

# Motivation

The real motivation behind Infinite-ISP is to streamline procedures, enhance current approaches, and develop advance algorithms in the ISP and computational photography field. This project provides a platform for relevant developers to work on the most exciting and challenging hardware-related issues that an ISP engineer encounters. The "open innovation" approach, enables anyone with the right skills, time, and interest to contribute generously to the Infinite-ISP project.

# Modules Description

Currently the model consists of the following 20 modules,

- Crop
- Dead Pixel Correction (DPC)
- Black Level Compensation (BLC)
- Optoelectronic Conversion Function (OECF)
- Digital Gain (DG)
- Bayer Noise Reduction (BNR)
- White Balance (WB)
- Auto White Balance (AWB)
- Demosaic
- Color Correction Matrix (CCM)
- Gamma Correction (GC)
- Auto Exposure (AE)
- Color Space Conversion (CSC)
- Local Dynamic Contrast Improvement
- Sharpening
- 2d Noise Reduction (2DNR)
- RGB Conversion (RGBC)
- Invalid Region Crop (IRC)
- Scale
- YUV Conversion Format (YUV 444 - 422)

Some functions like Local Lens Shading Correction (LSC), High Dynamic Range Imaging HDR Stitching, Tone Mapping (TM) will be added to the design in the future.

# Crop

The crop module is used to extract a rectangular region from an image by selecting an area at the center of the image with a specific size. This is useful for focusing on a specific portion of the image or more importantly removing unwanted regions from the edges.

## Algorithm Explanation

A straightforward cropping algorithm has been implemented, ensuring the preservation of the Bayer pattern. This is achieved by verifying that the number of rows and columns to be cropped are divisible by 4, and subsequently cropping an equal number of pixels from all sides of the image.

Cropping is applied on image only when both of the following conditions are met:

- Parameters [$\boldsymbol{new}$_height, $\boldsymbol{new}$_width] provided to the crop module are even numbers
- The difference between image dimensions and crop parameters is divisible by 4:

$$(image_\text{height} - \text{new_height})\ mod\ 4 = 0$$

$$(image_\text{width} - \text{new_width})\ mod\ 4 = 0$$

## Configuration Parameters

| Parameters | Details |
|---|---|
| $\boldsymbol{is}$_enable | When enabled it crops the image if the conditions listed above are met:<br>False: Disable<br>True: Enable |
| $\boldsymbol{new}$_width | New width of the input RAW image after cropping |
| $\boldsymbol{new}$_height | New height of the input RAW image after cropping |

Table 1: Crop configuration parameters

# Dead Pixel Correction

Image sensors sometimes come with manufacturing flaws that result in some malfunctioning pixels in the image. Despite advanced manufacturing techniques photo sensor arrays like CCD and CMOS sensor arrays include defective pixels because of noise, dust, or fabrication flaws. Hence identifying the defective pixels in the images and then replacing them with approximate values are two important phases of the DPC module.

Defective pixels can be categorized into two groups:

1. Dead: consistently low output
2. Hot: consistently high output

.

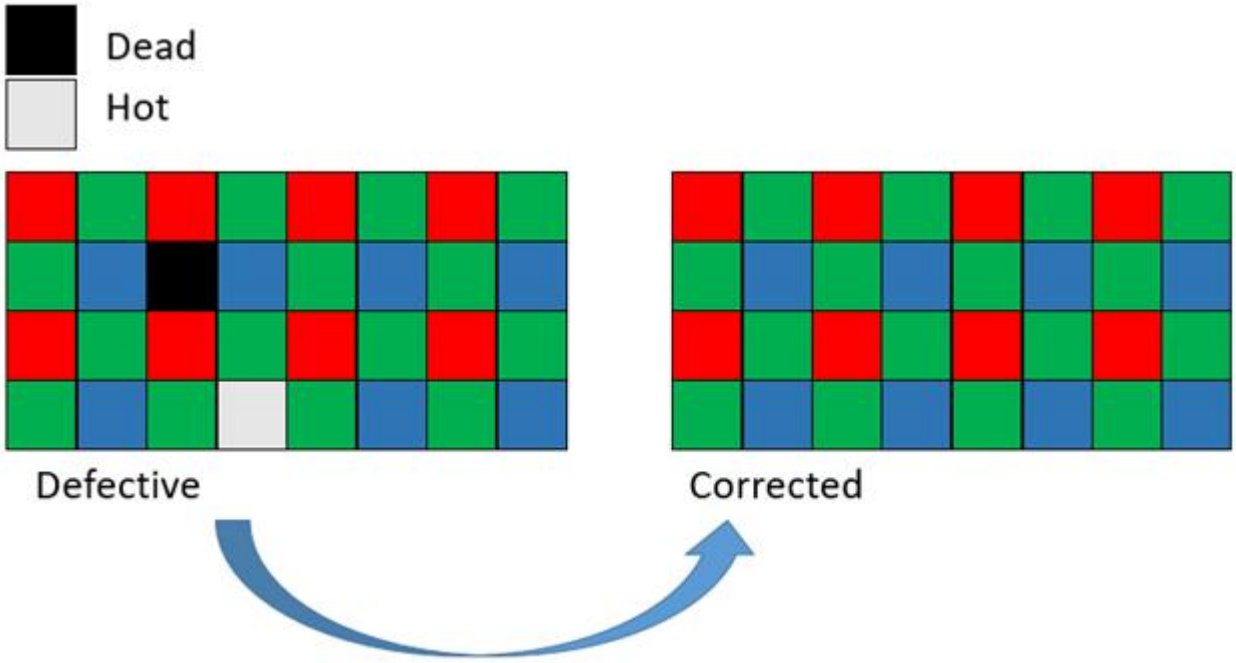



Figure 2: Defective and corrected pixels

# Algorithm Explanation

Each pixel of the image is tested by the DPC algorithm. To find defective pixels a $3 \times 3$ neighborhood of the *same* color channel surrounding the pixel being tested in a $5 \times 5$ window is created. The pixel is considered defective if it meets both of the following conditions:

1. The pixel value is less than the minimum pixel value of the all the pixels in the $3 \times 3$ neighborhood or greater than the maximum pixel value of all the pixels in the $3 \times 3$ neighborhood.
2. The difference between the pixel's value and each one of the pixels in the 3 x 3 neighborhood is greater than the programmed threshold parameter.

Once the pixel is identified as defective, its value is corrected by first computing four gradients (horizontal, vertical, left diagonal, and right diagonal) which pass through the pixel. The corrected value is then computed as the average of the two neighbors in the *minimum* gradient direction.

## Configuration Parameters

| Parameters | Details |
|---|---|
| ***is***_enable | When enabled, apply the DPC algorithm:<br>False: Disable<br>True: Enable |
| ***dp***_threshold | The threshold used for determining defective pixels. A low threshold results in more pixels being detected as dead and hence corrected |

Table 2: DPC configuration parameters

# Black Level Correction

This module is responsible for setting the image's pure black color. Setting the exposure time and various programmable gains (analog and digital) to their minimum values is one of the techniques to create a pure black image; however, even under these conditions, the resulting image may not be entirely black and as such the black level needs to be corrected. The four offset parameters, one for each channel, needed by this algorithm are generated by using the Tuning tool for a specific image sensor. Once the parameters have been generated, they are used by this module to correct the image's black level.

# Algorithm Explanation

To apply the black level correction, subtract the corresponding parameters from each of the pixels in the image. For a RGGB Bayer image the new values are computed as follows:

$$R^{'} = R - r_\text{offset}$$

$$Gr^{'} = Gr - gr_\text{offset}$$

$$Gb^{'} = Gb - gb_\text{offset}$$

$$B^{'} = B - b_\text{offset}$$

Additionally, by applying these offsets the range of values that a pixel generates is scaled down. This can optionally be corrected (enabled via the ***is***_linear parameter) by applying a linearization function over the desired range of values. Black level correction with linearization is computed as follows:

$$R^{'} = \frac{R - r_\text{offset}}{r_\text{sat} - \text{r_offset}}\ 2^{n}$$

$$Gr^{'} = \frac{Gr - gr_\text{offset}}{gr_\text{sat} - \text{gr_offset}}\ 2^{n}$$

$$Gb^{'} = \frac{Gb - gb_\text{offset}}{gb_\text{sat} - \text{gb_offset}}\ 2^{n}$$

$$B^{'} = \frac{B - b_\text{offset}}{b_\text{sat} - \text{b_offset}}\ 2^{n}$$

## Configuration Parameters

| Parameters | Details |
|---|---|
| $is$_enable | When enabled, apply the BLC algorithm:<br>False: Disable<br>True: Enable |
| $r$_offset | Red channel offset |
| $gr$_offset | Gr channel offset |
| $gb$_offset | Gb channel offset |
| $b$_offset | Blue channel offset |
| $is_linear$ | Enables or disables linearization. When enabled a linearization function is applied to scale the values over the desired range of values |
| $r$_sat | Red channel saturation level |
| $gr$_sat | Gr channel saturation level |
| $gb$_sat | Gb channel saturation level |
| $b$_sat | Blue channel saturation level |

Table 3: BLC configuration parameters

# Opto-Electronic Conversion Function—OECF

The relationship between the sensor output (photocell voltages) and incident light lux is called the optoelectronic conversion function (OECF). The sensor usually has an approximately linear response in that the detected signal is proportional to the incident light lux as imaged by the lens. Linearity is essential for achieving good white balance and color accuracy. The OECF module implements lookup curves for voltage re-mapping for a *specific* sensor. There are four lookup curves, one for each color channel which are generated through the Tuning tool.

## Algorithm Explanation

The Tuning tool uses ISO 14524 to generate the four curves. It uses a digital reflective camera contrast chart to plot density response vs. pixel voltage values.

## Configuration Parameters

| Parameters | Details |
|---|---|
| $\boldsymbol{is}$_enable | When enabled, applies the OECF curve:<br>False: Disable<br>True: Enable |
| $\boldsymbol{r}$_lut | The lookup table for the OECF curve. This curve is sensor dependent and is calculated by the Tuning tool |

Table 4: OECF configuration parameters

# Bayer Noise Reduction

The Bayer Noise Reduction (BNR) block suppresses noise in the Bayer domain before demosaicing. As a noise reduction block in the ISP pipeline, it is primarily tasked with reducing noise without effectively blurring the image details.

## Algorithm Explanation

For the BNR block, a Joint/Cross Bilateral Filter (JBF) has been selected as the appropriate candidate for RAW image denoising.

1. Green channel interpolation is performed on the input Bayer image to obtain a complete image of the green pixels using green interpolation kernels derived from the Malwar-He-Cutler algorithm.
2. Input Bayer image is deconstructed into R and B sub-images of half the size compared to the original Bayer image.
3. Interpolated G image based on a window (size determined by the file window parameter) is deconstructed into G pixels just at the R pixel locations. Additionally, the interpolated G image is also deconstructed into G pixels just at the B pixel locations. This produces R and B guided images.
4. JBF is applied on the R and B sub-images (produced in step 2) using their respective guide images (produced in step 3). JBF is also applied on the interpolated G image using itself as the guide image.
5. JBF uses two types of kernels i.e. range and spatial. A look-up table weighing scheme is created for the range kernel to facilitate the algorithm's hardware implementation.
6. The output Bayer image is reconstructed by joining the R, G, and B pixels from the JBF outputs (step 4).

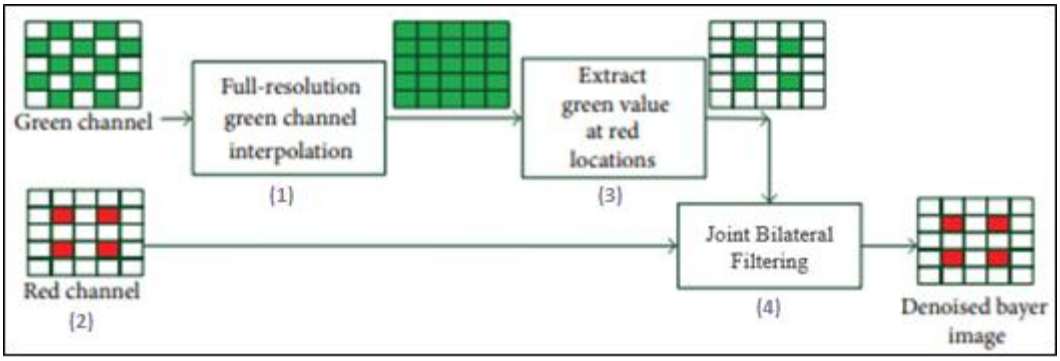



Figure 3: BNR algorithm

## Configuration Parameters

| Parameters | Details |
|---|---|
| ***is***_enable | When enabled, applies the BNR algorithm:<br>False: Disable<br>True: Enable |
| ***filt***_window | Filter size for BNR<br>This should be an odd window size e.g., 3 x 3, 5x5, etc. |
| ***r***_std_dev_s | Red channel Gaussian kernel strength<br>Higher strength values will result in increased blurring while a value of 0 is not permissible |
| ***r***_std_dev_r | Blue channel range kernel strength<br>Higher strength values will result in increased blurring while a value of 0 is not permissible |
| ***g***_std_dev_s | Gr and Gb Gaussian kernel strength.<br>Higher strength values will result in increased blurring while a value of 0 is not permissible |
| ***g***_std_dev_r | Gr and Gb range kernel strength<br>Higher strength values will result in increased blurring while a value of 0 is not permissible |
| ***b***_std_dev_s | Blue channel Gaussian kernel strength<br>Higher strength values will result in increased blurring while a value of 0 is not permissible |
| ***b***_std_dev_r | Blue channel range kernel strength<br>Higher strength values will result in increased blurring while a value of 0 is not permissible |

Table 5: BNR configuration parameters

# Digital Gain

Digital gain works with the auto exposure module to choose an appropriate constant value to be multiplied to all the channels to improve the image exposure.

## Algorithm Explanation

In the digital gain module, the raw image I is multiplied by an integer $g$, also known as Gain.

$$I^{'} = I * g$$

$$g = gain_\text{array}[\text{current_gain}]$$

An array of permissible gains, $\boldsymbol{gain}_\text{array}$, is defined in the configuration, and the index of the current gain is stored in the $\boldsymbol{current}_\text{gain}$ parameter, with its default value being zero. Gain selection for the Digital Gain module also depends on the $\boldsymbol{ae}_\text{feedback}$ parameter:

- $\boldsymbol{ae}_\text{feedback} = 0$ means image exposure is fine, so no change in gain is made.
- $\boldsymbol{ae}_\text{feedback} = \mathbf{1}$ means the image is overexposed, and the current gain is updated accordingly to decrease the gain.

$$current_gain = \begin{cases} current_gain, & current_gain = 0 \\ current_gain - 1, & otherwise \end{cases}$$

- $\boldsymbol{ae}_\text{feedback} = -1$ means the image is underexposed, and the current gain is updated accordingly to increase the gain.

$$current_gain = \begin{cases} current_gain, & current_gain = size(gain_array) \\ current_gain + 1, & otherwise \end{cases}$$

## Configuration Parameters

| Parameters | Details |
|---|---|
| $\boldsymbol{is}_\text{auto}$ | Flag to adjust digital gain according to AE Feedback<br>False : $\boldsymbol{current}_\text{gain}$ can only be changed manually<br>True : $\boldsymbol{current}_\text{gain}$ is adjusted according to $\boldsymbol{ae}_\text{feedback}$ |
| $\boldsymbol{gain}_\text{array}$ | List of permissible gains |
| $\boldsymbol{current}_\text{gain}$ | Index of the $\boldsymbol{gain}_\text{array}$ for the current gain which is being used. The index value starts at 0 |
| $\boldsymbol{ae}_\text{feedback}$ | AE feedback parameter<br>0 : Correct Exposure<br>1 : Overexposed |

| Parameters | Details |
| --- | --- |
| | -1 : Underexposed |

Table 6: DG configuration parameters

# White Balance

This module adjusts the white balance of raw images by applying appropriate gains, ensuring that white colors appear true to life. If gains are not available, they must first be determined through an auto-white balance process.

## Algorithm Explanation:

The white balance module selectively applies the provided gains to the red and blue channels, while the green channel remains unaffected. This process effectively restores color balance, making white colors appear accurate and natural.

$$R^{'} = R * r_\text{gain}$$

$$B^{'} = B * b_\text{gain}$$

## Configuration Parameters

| Parameters | Details |
|---|---|
| $\boldsymbol{is}_\text{enable}$ | Applies user-given white balance gains when enabled:<br>False: Disable<br>True: Enable |
| $\boldsymbol{is}_\text{auto}$ | Flag to adjust white balance gains according to AWB Algorithm<br>0 : $\boldsymbol{r}_\text{gain}$ and $\boldsymbol{b}_\text{gain}$ are applied on the raw image<br>1 : Gains from AWB module are applied |
| $\boldsymbol{r}_\text{gain}$ | Red channel gain. |
| $\boldsymbol{b}_\text{gain}$ | Blue channel gain. |

Table 7: WB configuration parameters

# 3A STATS

The 3A Stats module encompasses three essential aspects of the ISP (Image Signal Processing) pipeline: Auto White Balance (AWB), Auto Focus (AF), and Auto Exposure (AE). These 3A algorithms do not modify the image directly, but instead provide feedback in the form of 3A statistical parameters which are used by other pipeline modules. The AF module will be implemented in the future.

# Auto White Balance

AWB computes white balance gains as 3A statistics using the Grey World algorithm. In the White Balance (WB) module the AWB calculated gains are applied to the raw image.

# Algorithm Explanation

The AWB module calculates gains that adjusts the color balance in an image to restore the accurate colors of grey pixels. The Grey World algorithm is chosen for its optimal balance between ease of implementation in RTL (Register-Transfer Level) and overall performance.
There are two main steps of the AWB module:

1. Preprocess Pixel Filtering
2. Gain Calculation using Grey World Algorithm

## Preprocess Pixel Filtering

Before calculating white balance gains, a pre-processing pixel filtering step is performed, in which a percentage of underexposed and overexposed pixels are removed from the image. The percentage of underexposed and overexposed pixels is provided through AWB configuration parameters (***underexposed***_percentage, ***overexposed***_percentage). These pixels are not used in the next step.

## Grey World Algorithm

The Grey World algorithm is a white balance method based on the assumption that, on average, a scene's color is neutral grey. This assumption holds when there is a balanced distribution of colors in the scene. Given this balanced distribution, the average reflected color represents the color of the light source. To estimate the color cast of the illumination, the algorithm compares the average color to grey. The Grey World algorithm calculates an illumination estimate by computing the mean of each image channel

$$C_{avg} = \frac{\sum_{n=0}^{N} C(n)}{N}$$

$C \in \{R, G. B\}$ and $N$ is the total number of pixels in the image. White balance gains for red and blue channels are calculated as shown below. These gains are then applied to the corresponding channels of the next frame.

$$R_{gain} = \frac{G_{avg}}{R_{avg}}$$

$$B_{gain} = \frac{G_{avg}}{B_{avg}}$$

## Configuration Parameters

| Parameters | Details |
|---|---|
| ***is***_enable | Calculate white balance gains of the image when enabled:<br>False: Disable<br>True: Enable |
| ***stats***_window_offset | Specifies crop dimensions to obtain a stats calculation window<br>Should be an array of element, [Up, Down, Left, Right]<br>Should be a multiple of 4 |
| ***underexposed***_percentage | [0 - 100] - Set % of dark (underexposed) pixels to exclude before AWB gain calculation |
| ***overexposed***_percentage | [0 - 100] - Set % of saturated (overexposed) pixels to exclude before AWB gain calculation |

Table 8: AWB configuration parameters

# Auto Exposure

AE is one of the three modules of 3A Control. Auto-exposure (AE) is a crucial feature in consumer digital cameras. AE generates a parameter based on the evaluation of the current image to auto-adjust the camera for the next frame[1]. This parameter is then used by the Digital Gain module.

*High-end cameras primarily control scene brightness using exposure, aperture, and analog gain. However, video cameras have a lower exposure limit due to the required frame rate. Additionally, economical cameras often have limited sensor control, so AE control settings are mainly adjusted through gains (Analog, Digital, and ISP gain, which together contribute to the scene's ISO).* In the model AE feedback is used by Digital Gain module to adjust image brightness.

# Algorithm Explanation

The primary AE stat for determining scene brightness is the skewness of the scene's luminance histogram. The skewness of the luminance histogram around a central luminance provides insight into whether an image is underexposed or overexposed.

## Skewness

Moments are widely used in image analysis since they can derive invariants related to specific transformation classes. A moment is a quantitative measure of the shape of a set of points. The nth moment about zero of a probability density function $f(x)$ is the expected value of $x(n)$, referred to as a raw moment. The moments about the mean (with C being the mean) are called central moments, which describe the shape of the function independently of translation.

The nth moment of a real-valued continuous function $f(x)$ of a random variable $x$ about a value $C$ is given by

$$\mu_n = \int_{-\infty}^{\infty} (x - C)^n f(x) dx$$

Central moments are typically used for the second and higher moments, as they offer more precise information about the distribution's shape. The first central moment $\mu_1$ is mean, and the second central moment $\mu_2$ is the variance. The third central moment is skewness that is defined as a measure of a distribution's lopsidedness. A symmetric distribution has a skewness of zero, while a negatively skewed distribution has a longer tail on the left and a positively skewed distribution has a longer tail on the right.

[1] Note that images are continuously being evaluated as they flow through the pipeline and the pipeline parameters are being adjusted till the point the image is actually taken.

If a grayscale distribution is uniform, it has a high dynamic range, high contrast, and produces a clear image. A uniform grayscale distribution is also symmetric, with a third central moment equal to zero. Experimental results show that as a grayscale distribution becomes more uniform, the skewness of the histogram approaches zero, especially for images with non-bimodal histograms. Consequently, the skewness of a histogram is used to evaluate the uniformity of the histogram.

In the model digital gain is applied to an image to correct the histogram skewness. The skewness as defined above using the central moment equation results in an excessively large value. So, the skewness is calculated using Fisher Pearson coefficient of skewness:

$$skewness = \frac{\mu_3}{{\mu_2}^{\frac{3}{2}}}$$

Where :

$$\mu_n = \int_{-\infty}^{\infty} (x - C)^n f(x) dx$$

$C$ is provided my AE configuration parameter ***center***_illuminance. $skewness$ is calculated above along with ***histogram***_skewness defines the status of the image exposure settings:

- $|skewness| \leq$ ***histogram***_skewness: Image has corrected exposure
- $skewness < -$ ***histogram***_skewness: Image is underexposed
- $skewness >$ ***histogram***_skewness: Image is overexposed

## Configuration Parameters

| Parameters | Details |
|---|---|
| ***is***_enable | When enabled, apply the 3A - Auto Exposure algorithm:<br>False: Disable<br>True: Enable |
| ***stats***_window_offset | Specifies crop dimensions to obtain a stats calculation window<br>Should be an array of element, [Up, Down, Left, Right]<br>Should be a multiple of 4 |
| ***center***_illuminance | Pixel Values around which skewness is calculated<br>The value of center illuminance for skewness calculation ranges from 0 to 255 |
| ***histogram***_skewness | Along with the *skewness* calculated above, it defines the exposure settings |

Table 9: AE configuration parameters

# Color Filter Array

The CFA module is an integral component of the ISP pipeline that transforms the black-level corrected, white-balanced, linearized 2D image into a 3D RGB image format.

A Color Filter Array (CFA) is positioned in front of the sensor to capture color information. This array typically made of three filters, restricts the sensitivity of each photocell to one part of the visible spectrum. As a result, each pixel of the CFA image only contains information about this limited range, i.e., one color response. However, to render images on a specific display device, three colors per pixel are required. Demosaicing is therefore employed to reconstruct the missing colors at each pixel location.

## Algorithm Explanation

After thorough analysis, Malvar-He-Cutler demosaicing algorithm was selected (shown in Figure 4). Image demosaicing, or demosaicing, is the interpolation problem of estimating complete color information for an image captured through a Color Filter Array (CFA), particularly using the Bayer pattern. Demosaicing is achieved by convolving the image with a set of linear filters. There are eight different filters shown in Figure 5Figure 5 for interpolating the various color components at different locations.

The CFA module is implemented in the pipeline after DPC, BLC, and WB. Instead of using a constant or near-constant hue approach, the algorithm uses the criterion of edges to have much stronger luminance than chrominance components.

In the Malvar-He-Cutler algorithm, to interpolate a green value at a red pixel location, the algorithm compares the red pixel information with its estimate of a bilinear interpolation of the nearest red samples. If it differs from that estimate, it means there is a sharp luminance change at that pixel. Consequently, the algorithm bilinearly interpolates the green value by adding a portion of this estimated luminance change. This method is a gradient-corrected bilinear interpolation approach and its flowchart is shown below:

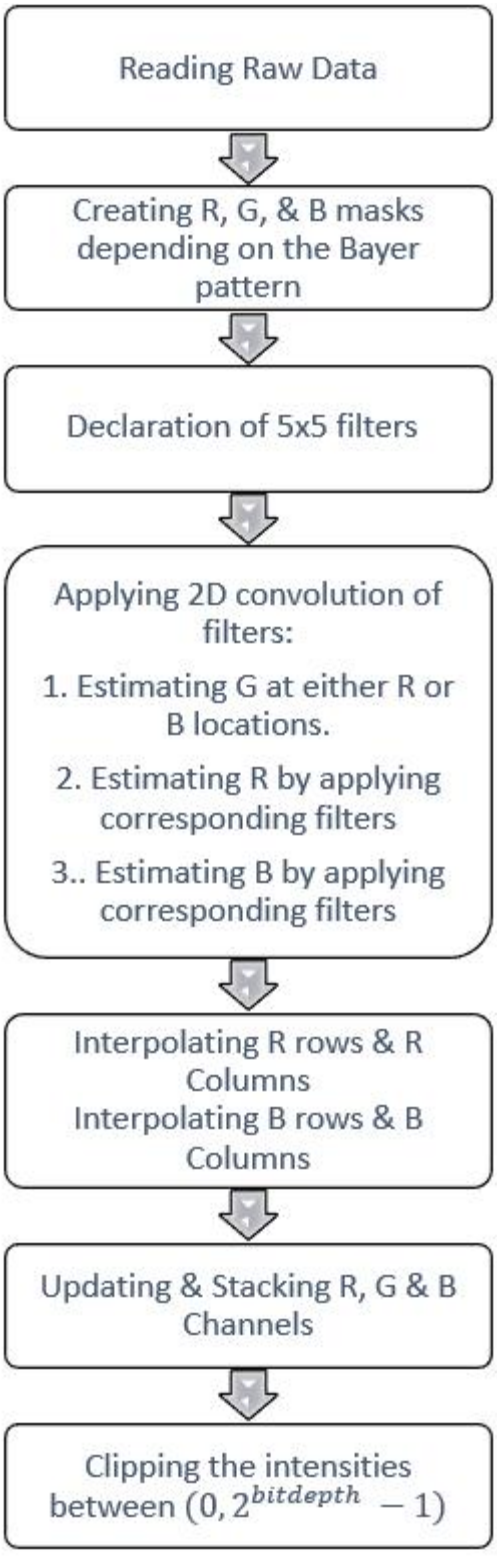



Figure 4: Flowchart for the Malwar He Cutler's Demosaicing Algorithm

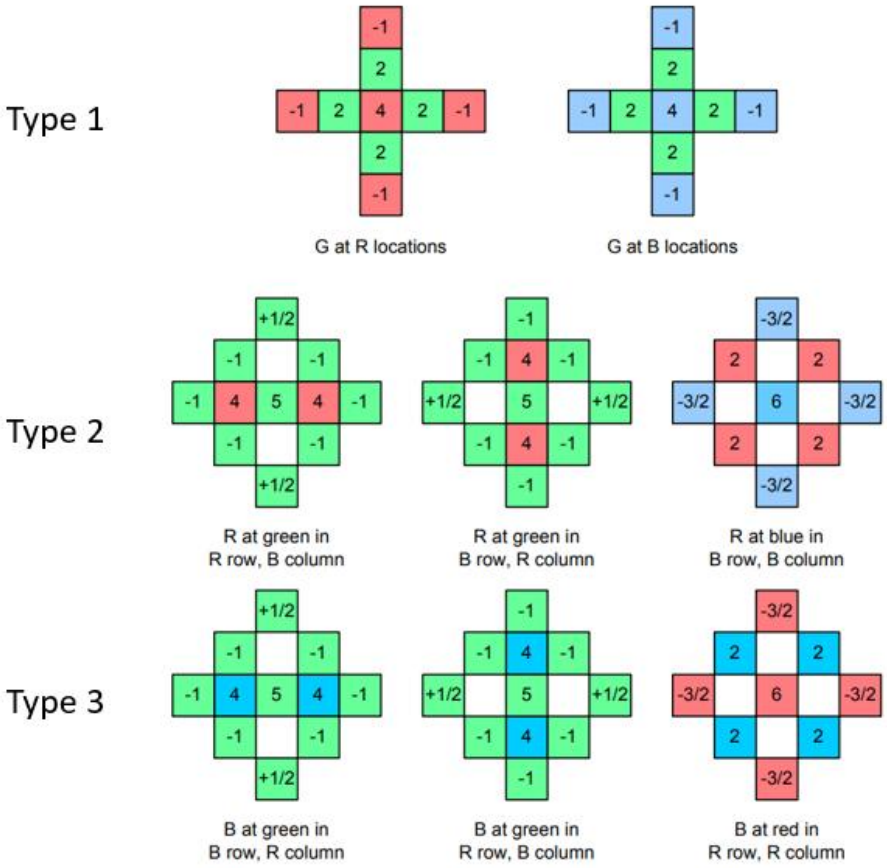



Figure 5: Filter Coefficients for linear interpolation of R, G & B data.

The Malvar-He-Cutler demosaicing algorithm is explained below:

1. **Mask Generation:** From a raw Bayer image shown in Figure 6 three r, g, and b masks are generated shown in Figure 7.

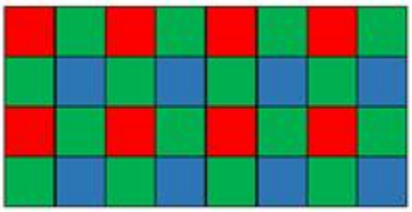


Figure 6: Raw Image with RGGB Bayer pattern

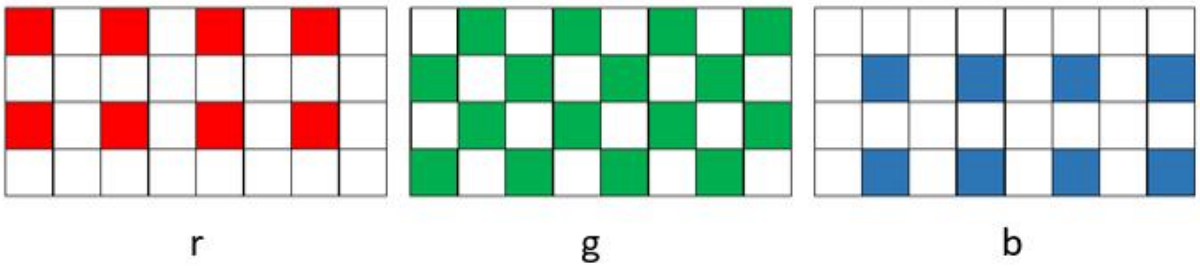



Figure 7: Masking channels

**G Channel Extraction**

1. On G channel, g pixels at r and b locations are estimated using the result of the convolution of filters from type 1 in Figure 5 with the raw image as shown below.

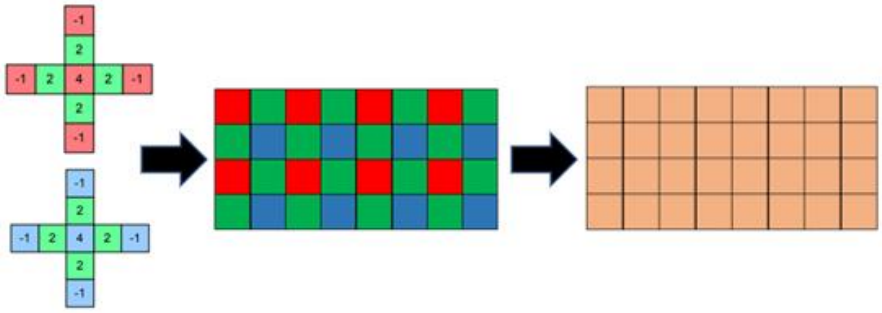


Figure 8: Convolution of a raw image with Filter Type 1

2. Result of step 1 is masked through r and b masks to get g pixels value at r and b location as shown below:

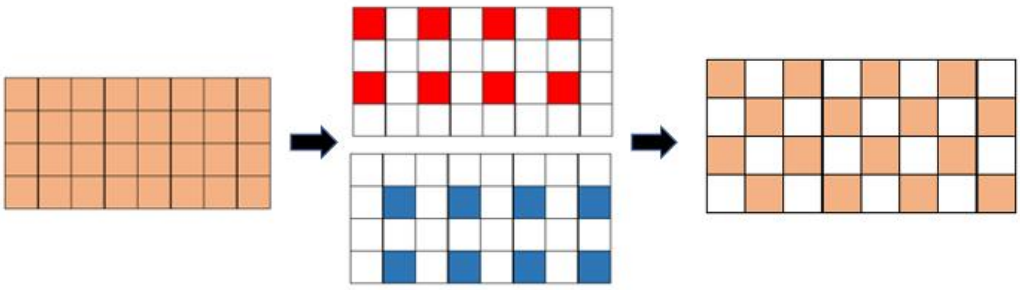


Figure 9: Estimation of g at r and b locations

3. Interpolated g values of Step 2 are added at the respective r and b location to obtain a final G channel:

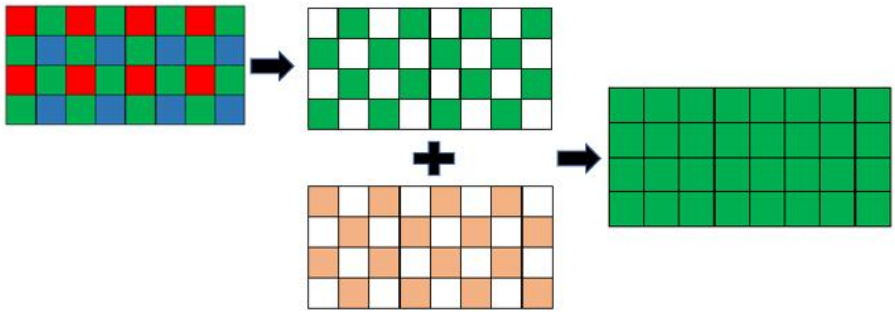


Figure 10: Final G channel

**R Channel Extraction**

1. Identify three locations where R has to be estimated as shown below, essentially creating three masks:

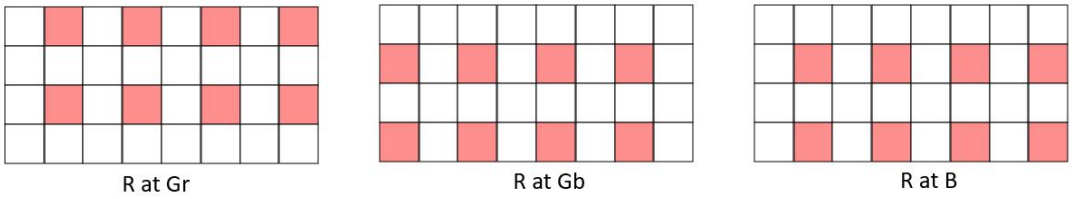



Figure 11: Location of R pixels masks

2. Convolve the raw image with each of the three types 2 filters shown in Figure 5 to produce three R estimated images.

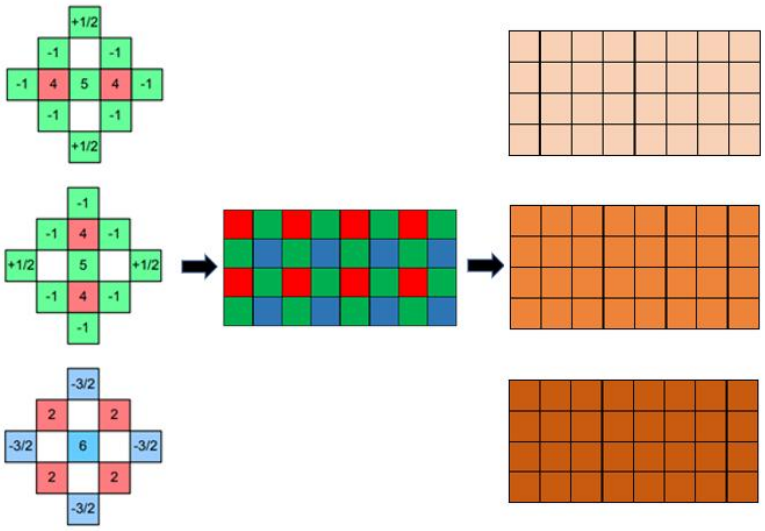



Figure 12: Convolution of Type 2 filters with raw image for R estimated images

3. Use the estimated R images produced in Step 2 along with the masks produced in Step 1 to get the estimated R channel values.

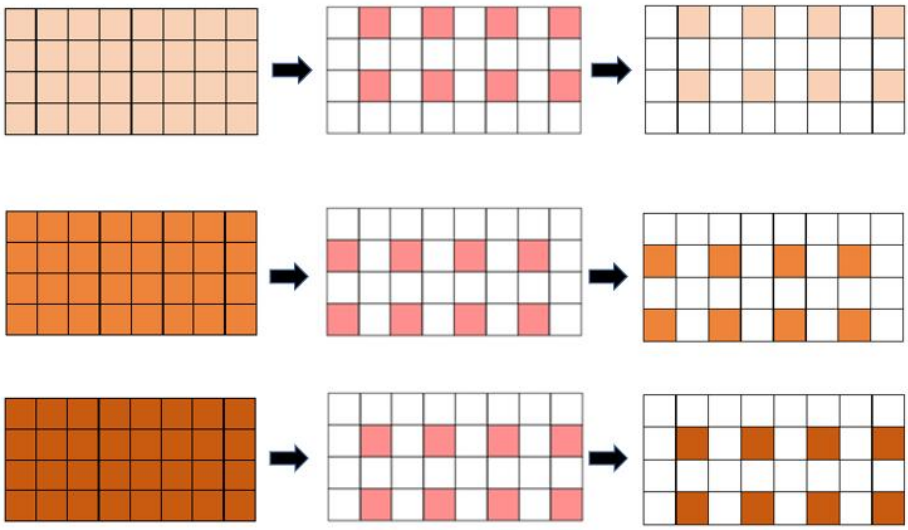


Figure 13: Extracting estimated R values

4. Estimated R channels values from step 3 together with R channel values of raw image produce the final R channel.

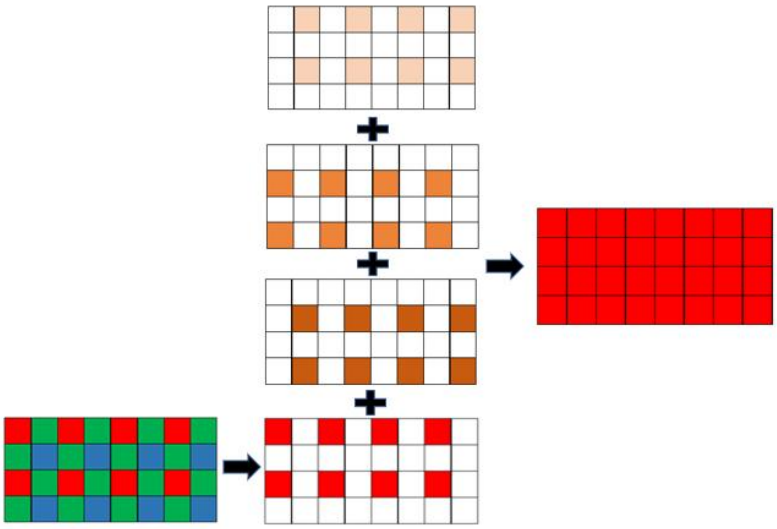


Figure 14: Final R Channel

**B Channel Extraction**

For B channel extraction the same procedure as for the R channel is followed. At Step 1 estimating locations are now B at Gb, B at Gr and B at R where B channel has to be estimated and filters from type 3 will be applied in Step 2.

## Configuration Parameters

CFA has no configuration parameters and is always active in the pipeline.

# Color Correction Matrix

To optimize color output in digital images, it is essential to address factors such as optical spectral properties (including lenses and filters), a variety of lighting sources (such as tungsten, fluorescent, and daylight), and the characteristics of the sensor's color that can lead to color inaccuracies in the captured image. The Color Correction Matrix (CCM) module plays a critical role in counteracting the effects of incorrect color representation and applies a color correction matrix to the image, ensuring a more precise display of colors as captured by the source imaging system.

## Algorithm Explanation

The Color Correction Matrix (CCM) is a 3x3 matrix specifically designed to adjust the Red, Green, and Blue (RGB) color channels of an image. Each component within the matrix acts as a scaling factor, modifying the color values of each channel based on a calculated combination of the original values. This matrix is systematically applied to every pixel within the image, effectively transforming the color values in accordance with the given formula:

$$\begin{bmatrix} R' \\ G' \\ B' \end{bmatrix} = CCM * \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

$$\begin{bmatrix} R' \\ G' \\ B' \end{bmatrix} = \begin{bmatrix} RR & RG & RB \\ GR & GG & GB \\ BR & BG & BB \end{bmatrix} * \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

## Configuration Parameters

| Parameters | Details |
|---|---|
| ***is***_enable | When enabled, the user given 3x3 CCM is applied to the 3D RGB image with rows sum to 1 convention:<br>False: Disable<br>True: Enable |
| ***corrected***_red | Row 1 of CCM |
| ***corrected***_green | Row 2 of CCM |
| ***corrected***_blue | Row 3 of CCM |

Table 10: CCM configuration parameters

# Gamma Correction

Gamma correction alters the input image by translating it to a nonlinear space. It matches the nonlinear response of display devices and broadens or compresses the dynamic range of images.

## Algorithm Explanation

A look-up table is a common method for implementing gamma correction in hardware and is simple to modify and encode.

## Configuration Parameters

| Parameters | Details |
|---|---|
| ***is***_enable | When enabled, applies tone mapping gamma using the look-up table:<br>False: Disable<br>True: Enable |
| ***gamma***_lut_8 | The look-up table for the gamma curve for 8-bit image |
| ***gamma***_lut_10 | The look-up table for the gamma curve for 10-bit image |
| ***gamma***_lut_12 | The look-up table for the gamma curve for 12-bit image |
| ***gamma***_lut_14 | The look-up table for the gamma curve for 14-bit image |

Table 11: GC configuration parameters

# Color Space Conversion

The color space conversion transforms the RGB color representation of an image to another color space. Different color spaces have been designed to accommodate varying proportions of chroma and luma. Hardware displays typically utilize the YUV color space. This choice is primarily rooted in the history of television technology, where black and white televisions relied solely on a single luminance channel. As color television emerged, the U and V channels were introduced to transmit color information, while the Y channel continued to convey luminance, thus ensuring backward compatibility with earlier black and white televisions.

# Algorithm Explanation

In the model, the color space conversions are based on ITU-R standards. There are three ITU standards for YUV conversion.

1. BT.601 – for SDTV
2. BT.709 – for HDTV

We have implemented BT.601. BT.709 while BT.2020 will be implemented in the future.

## BT.601

The conversion formula uses the ITU-R standard, multiplying the $YUV_{mat}$ to the RGB image.

$$YUV_{mat} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ -0.1687 & -0.3313 & 0.5 \\ 0.5 & -0.4187 & -0.0813 \end{bmatrix}$$

$$\begin{bmatrix} Y \\ U \\ V \end{bmatrix} = YUV_{mat} * \begin{bmatrix} R_{norm} \\ G_{norm} \\ B_{norm} \end{bmatrix}$$

The conversion requires a normalized RGB image, and the output YUV has a Y range from 0 to 1, whereas U and V range from -0.5 to 0.5. This conversion is not ideal for RTL implementation due to the multiplication involving floating-point numbers. Therefore, $YUV_{mat}$ is converted into an integer matrix, also known as YCrCb, to enable digital conversion. The process for converting RGB to $YC_rC_b$ using the $YC_rC_{b_{mat}}$ involves the following steps:

1. Multiply $YUV_{mat}$ with $2^m$, where $m$ is the matrix precision bits to get $YC_rC_{b_{mat}}$. The higher the bit value, the higher the precision, resulting in multiplication with larger bit depths. $m$ can only be an even integer from 8 to 16. For the model, m is set to 8.

$$YC_rC_{b_{mat}} = YUV_{mat} * 2^m$$

$$YC_rC_{b_{mat}} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ -0.1687 & -0.3313 & 0.5 \\ 0.5 & -0.4187 & -0.0813 \end{bmatrix} * 2^m$$

$$YC_rC_{b_{mat}} = \begin{bmatrix} 77 & 150 & 29 \\ 131 & -110 & -21 \\ -44 & -87 & 138 \end{bmatrix}$$

2. Multiply RGB with the $YC_rC_{b_{mat}}$ and add a digital offset to correct the black level of the YCrCb image. Divide the result of this matrix multiplication by $2^m$ to obtain a YCrCb image with the same bit depth as the input RGB image. Then, add a digital offset matrix to correct the black level of the $YC_rC_b$ image.

$$\begin{bmatrix} Y \\ C_r \\ C_b \end{bmatrix} = \frac{\left( YC_rC_{b_{mat}} * \begin{bmatrix} R \\ G \\ B \end{bmatrix} \right)}{2^m} + \begin{bmatrix} Y_{offset} \\ C_{r_{offset}} \\ C_{b_{offset}} \end{bmatrix}$$

3. This offset matrix depends on bit depth of $YC_rC_b$ image $n$ and it can be defined as:

$$\begin{bmatrix} Y_{offset} \\ C_{r_{offset}} \\ C_{b_{offset}} \end{bmatrix} = \begin{bmatrix} 2^{\frac{n}{2}} \\ 2^{n-1} \\ 2^{n-1} \end{bmatrix}$$

### BT.709

BT.709 conversion is similar to BT.601 with different conversion matrix:

$$YUV_{mat} = \begin{bmatrix} 0.2126 & 0.7152 & 0.0722 \\ -0.1147 & -0.3854 & 0.5 \\ 0.5 & -0.4542 & -0.0458 \end{bmatrix}$$

This decimal matrix also known as analog matrix is also converted into digital matrix in the same way described in BT. 601 section following same steps to achieve the final $YC_rC_b$ Image.

$$YC_rC_{b_{mat}} = \begin{bmatrix} 47 & 157 & 16 \\ -26 & -86 & 112 \\ 112 & -102 & -10 \end{bmatrix}$$

Note: In all the module after the CSC module, $YC_rC_b$ image is referred as YUV image.

## Configuration Parameters

| Parameters | Details |
|---|---|
| ***conv***_standard | Set standard to be used for conversion<br>1 : Bt.709 HD<br>2 : Bt.601/407 |

Table 12: CSC configuration parameters

# Local Dynamic Contrast Improvement

The Local Dynamic Contrast Improvement (LDCI) module applies Contrast Limited Adaptive Histogram Equalization (CLAHE) to the luma (Y) channel to improve local contrast. It runs after Color Space Conversion and before Sharpening; chroma (Cb/Cr) passes through unchanged.

## Algorithm Explanation

LDCI works in two steps:

1. Per-tile histogram equalization

2. Bilinear blending across tile boundaries

The frame is divided into $wind \times \mathrm{wind}$ tiles (wind is window size). For each tile independently, a 256-bin histogram of its Y values is built, and a contrast-enhancement lookup table (LUT) is computed from that tile's own local histogram via CLAHE. The bin counts above clip_limit are clipped (to avoid over-amplifying noise in flat regions), and the clipped-off count is redistributed evenly across all 256 bins before the LUT is built from the cumulative distribution. Each tile ends up with its own LUT, tuned to its own local contrast.

To avoid visible seams at tile boundaries, each output pixel is produced by bilinearly blending the LUTs of its neighbouring tiles. A tile in a frame corner uses only its own LUT (no blending), a tile on a frame edge blends with one neighbour, and an interior tile blends with its four surrounding tiles.

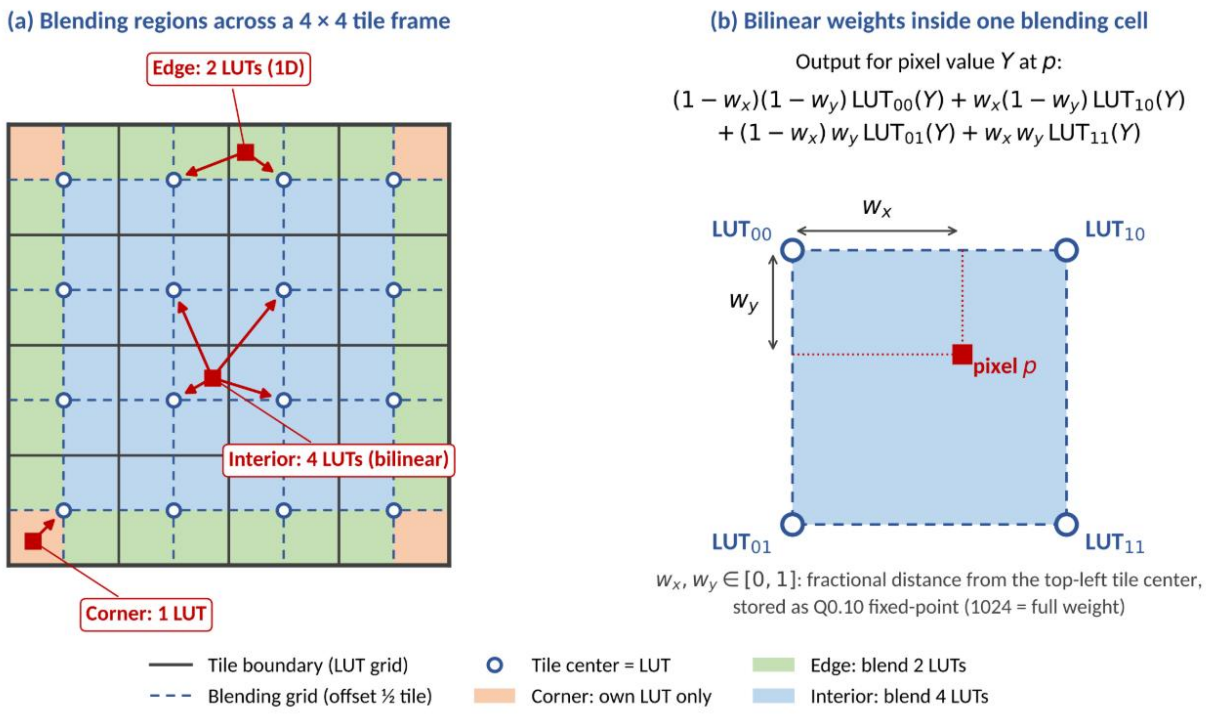



Figure 15: LDCI Tile boundary blending

# Configuration Parameters

| Parameters | Details |
|---|---|
| ***isEnable*** | When enabled, applies the LDCI to the image<br>False: Disable<br>True: Enable |
| ***clip_limit*** | Per-bin histogram clip threshold for a wind x wind tile (clamped to 8% of wind if set at or above wind) |
| ***wind*** | Tile size in pixels (each tile is wind x wind) |

Table 13:LDCI configuration parameters

# Sharpening

The Sharpen module amplifies the selected high-frequency band of the image in order to improve the edges and texture details of the image. The strength of the applied sharpening and the band selection are dependent on the 2DNR module in the pipeline to ensure high visual fidelity.

## Algorithm Explanation

The Unsharp Masking uses the luma component of the image. In the first step, we convolve the luminance $Y$ component of the input image with a Gaussian filter $G(\sigma)$ where $\sigma$ is the standard deviation value. Next, we use this attained low-frequency component to extract the remaining higher frequencies $H$ from the image in the following way:

$$H = Y - (G(\sigma) * Y)$$

The extracted high frequencies of then enhanced by the given strength $k$ and then added back into the original luminance value in the following way to achieve the sharpened luma $Y_{shrp}$:

$$Y_{shrp} = Y + k \cdot H$$

## Configuration Parameters

| Parameters | Details |
|---|---|
| ***isEnable*** | When enabled, applies the sharpening to the image<br>False: Disable<br>True: Enable |
| ***sharpen_sigma*** | [1, 10] – Parameter to define the Standard Deviation of the Gaussian Filter |
| ***sharpen_strength*** | [0.1, 2] – Parameter controls the sharpen strength applied on the high frequency components |

Table 14: Sharpening Configuration Parameters

# 2D Noise Reduction

A denoising tool, 2D Noise Reduction (2DNR), is incorporated into the system to minimize the impact of image noise, which deteriorates image quality. In conducting a literature review, the behavior of various spatial domain denoising methods was analyzed, along with the effects caused by adaptively chosen weights associated with them. Taking into account the hardware and computational constraints, we selected the Non-Local Mean (NLM) algorithm for denoising images.

# Algorithm Explanation

## Non-Local Means Filter

The NLM algorithm is a spatial domain denoising technique. These types of methods can be further categorized as local and non-local filters. Natural images exhibit self-similarity, and the NLM filter leverages this characteristic. NLM denoises an image by taking the weighted average values of all similar pixels throughout the entire image.

The weights assigned to these similar pixels are computed based on the Euclidean distance between the intensities of the pixels. They are sorted in decreasing order to assign more weight to the most similar pixel in the image. This concept is founded on the idea that, for denoising a single pixel in an image, the weighted average is computed using all those pixels in the entire image, which share similar intensity or grey levels, and are close to each other based on their Euclidean distance.

By considering the spatial relationships between pixels and their intensities, the NLM filter effectively reduces noise while preserving important image details. However, it may be more computationally intensive than other denoising techniques due to its non-local approach.

Mathematically, we can represent this as:

$$I_{denoised}(x) = \frac{1}{C(x)} \sum_{y \in \Omega} I(y).W(x,y)$$

$$W(x,y) = \exp\left(-\frac{\left||I(x) - I(y)|\right|^2}{h^2}\right)$$

$I_{denoised}(x)$, is the denoised intensity value at x.

$C(x)$, is the normalization factor to ensure that the weights sum up to 1.

$\Omega$, represents the search window containing all pixels in the image.

$W(x,y)$, is the weight assigned to each pixel in the search window based on its similarity with the centered pixel.

$h$, is the strength parameter that controls the strength of the denoising effect by influencing the assignment of weights.

$I(y)$, represents all the pixels placed in the search window.

## Configuration Parameters

| Parameters | Details |
|---|---|
| ***is***_enable | When enabled, apply the non-local mean filtering:<br>False: Disable<br>True: Enable |
| ***window***_size | Search window size for NLM filter.<br>It should be odd<br>9 is set to avoid the computational cost. |
| ***wts*** | 0-100] Strength parameter for NLM. The higher the value, the higher will be the smoothing or blurring<br>Set to 5 |

Table 15: 2DNR configuration parameters

# RGB Conversion

RGB conversion module determines the output format of the model. If enabled, it performs inverse color space conversion from YUV to RGB based on conversion type specified in CSC module parameter otherwise the pipeline outputs a YUV image.

The conversion formula uses the ITU-R standard for BT-601 and BT_709, multiplying the $RGB_{mat}$ to the YUV image.

$$RGB_{mat} = {YUV_{mat}}^{-1}$$

$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = RGB_{mat} * \begin{bmatrix} Y \\ U \\ V \end{bmatrix}$$

$RGB_{mat}$ matrix is a decimal; therefore, it is converted into an integer matrix using the following steps:

1. Multiply $YUV_{mat}$ with $2^m$, where $m$ is the matrix precision bits to get ${YC_rC_b}_{mat}$. The higher the bit value, the higher the precision, resulting in multiplication with larger bit depth $m$ can only be an even integer from 8 to 16. For the model, m is set to 8.

$$digital_\mathrm{RGB}_{mat} = RGB_{mat} * 2^m$$

$$digital_\mathrm{RGB}_{mat} = \begin{bmatrix} 74 & 0 & 144 \\ 74 & -13 & -34 \\ -74 & 135 & 0 \end{bmatrix}$$

2. Subtract the digital offset and multiply YUV image with the $digital_\mathrm{RGB}_{mat}$. Divide the result of this matrix multiplication by $2^m$ to obtain a RGB image with the same bit depth as the input RGB image.

$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \frac{digital_\mathrm{RGB}_{mat} * \left( \begin{bmatrix} Y \\ U \\ V \end{bmatrix} - \begin{bmatrix} Y_{offset} \\ C_{r_{offset}} \\ C_{b_{offset}} \end{bmatrix} \right)}{2^m}$$

3. This offset matrix depends on bit depth of YUV image bit depth $n = 8$ and it can be defined as:

$$\begin{bmatrix} Y_{offset} \\ C_{r_{offset}} \\ C_{b_{offset}} \end{bmatrix} = \begin{bmatrix} 2^{\frac{n}{2}} \\ 2^{n-1} \\ 2^{n-1} \end{bmatrix}$$

## Configuration Parameters

| Parameters | Details |
|---|---|
| ***is***_enable | When enabled, converts output YUV image to RGB<br>False : Disable – Output image format is YUV<br>True : Enable – Output Image format is RGB |

Table 16: RGBC configuration parameters

# Invalid Region Crop

The Invalid Region Crop (IRC) module functions in unison with the scaling module to ensure images are cropped to a valid size, specifically 1920x1080 or 1920x1440. This step is essential for the utilization of a hardware-friendly scaling approach, which supports only the specified input sizes. It's critical to note that if the IRC module is deactivated, the ***is***_hardware flag in the Scale module should also be switched off. This adjustment maintains the consistent and error-free execution of the scaling operations.

## Configuration Parameters

| Parameters | Details |
|---|---|
| ***is***_enable | When enabled, scales down the input image.<br>False: Disable<br>True: Enable |
| ***crop***_to_size | Only two permissible values to set crop size:<br>1 : crops the image to size 1920x1080<br>2 : crops the image to size 1920x1440 |
| ***height***_start_idx | Starting height-index for crop |
| ***width***_start_idx | Starting width-index for crop |

Table 17: IRC configuration parameters

# Scale

The Scale block is implemented as one of the final blocks in the ISP pipeline to downscale a full-resolution image to a supported resolution/size per the user's requirements. The Scale block is generally implemented in IP camera ISPs to create multiple channels/streams for storage and transmission in surveillance cameras.

Scaling only preserves the aspect ratio (the ratio between image height and width $\frac{h}{w}$) when height and width are scaled by the same factor $\frac{h*c}{w*c} = \frac{h}{w}$. Cropping does not preserve the aspect ratio even if same amount is cropped from both height and width $\frac{h-c}{w-c} \neq \frac{h}{w}$.

Up/Down scaling height and width at the same time is equivalent to scaling the height first and width second or vice versa. For example, downscaling with Nearest Neighbor method using a square $2 \times 2$ window is equal to downscaling by $2 \times 1$ window followed by downscaling with $1 \times 2$ window.

# Algorithm Explanation

- Input image: 3D array (RGB or YUV image) of size $1920 \times 1080$ or $1920 \times 1440$.
- Output image: Downscaled 3D array (RGB or YUV image). The valid output sizes corresponding to each input size are mentioned in Table 18.

The image is scaled using the following steps:

1. Down-scale the image by the integer scaling factor using the nearest neighbor method.
2. Crop the image to required output-size if needed.
3. The algorithm scales each channel one by one using the same steps.

The table below shows the hand-picked scaling factors and crop values that have been used in the design. These values are hardcoded in the code for each input size using classes.

| Input Size | Output size | Downscale factor | | Crop value | |
|---|---|---|---|---|---|
| | | width | Height | width | Height |
| **$1920 \times 1080$** | $640 \times 480$ | 3 | 2 | – | 60 |

| Input Size | Output size | Downscale factor | | Crop value | |
|---|---|---|---|---|---|
| | | width | Height | width | Height |
| **1920 × 1080** | 640 × 360 | 3 | 3 | – | – |
| **1920 × 1440** | 960 × 720 | 2 | 2 | – | – |
| **1920 × 1440** | 640 × 480 | 3 | 3 | – | – |

Table 18: Scaling – Valid Output Sizes

## Configuration Parameters

| Parameters | Details |
|---|---|
| ***is***_enable | When enabled, scales down the input image<br>False: Disable<br>True: Enable |
| ***new***_width | Downscaled width of the output image |
| ***new***_height | Downscaled height of the output image |

Table 19: Scale configuration parameters

# YUV Format – 444-422

The YUV conversion format is a way of subsampling YUV images to save the bandwidth. Subsampling of YUV is possible because the human eye is less sensitive to color differences compared to luminance differences, allowing for subsampling of the $C_rC_b$ channel without significantly affecting the output. That is why no subsampling is done for the Y channel.

## Algorithm Explanation

In YUV format, the Y, U, and V components are stored in a single array, known as the packed format. Pixels are organized into macro pixel groups, with the layout depending on the YUV format. Each YUV format described has an assigned FOURCC code, which is a 32-bit unsigned integer created by concatenating four ASCII characters:

- Y : Luma
- U : Blue-Difference
- V : Red-Difference
- A : Transparency

YUV formats implemented in the model are

1. 4:4:4
    a. AYUV
2. 4:2:2
    a. YUYV
    b. UYVY

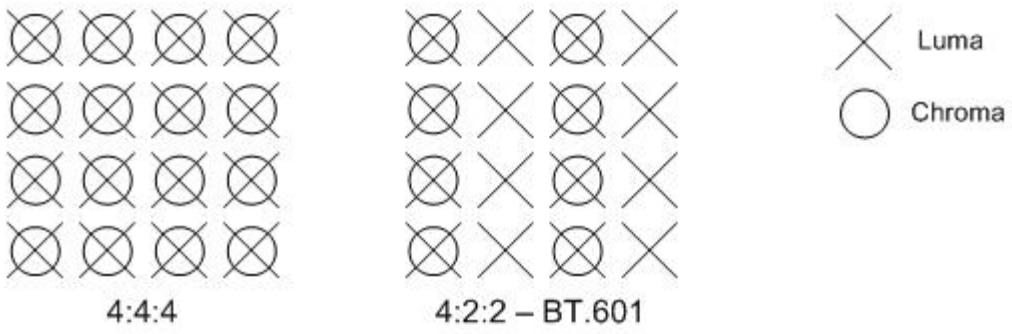



Figure 16: Pictorial representation of 4:4:4 and 4:2:2

### 4:4:4 Format

In this format, no subsampling is done, and YUV is saved in the following manner:

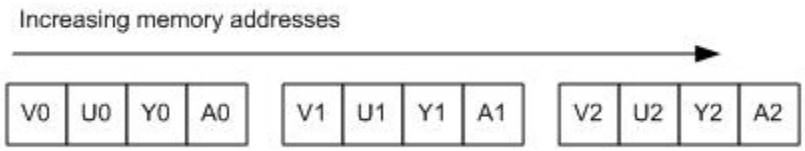



Figure 17: Memory view of bytes for packed format

The recommended FOURCC format for 4:4:4 is shown above. In FOURCC, each memory element is 32 bits with 4 bytes, each representing either Y, U, or V based on the chosen formatting. In the model, a simple 4:4:4 format is implemented, where each memory chunk has 3 bytes of Y, U, or V. Since there is no information regarding the alpha channel yet, it is not considered in this conversion.

| Pixel Number | Pixel Values |
|---|---|
| 0 | U0Y0V0 |
| 1 | U1Y1V1 |
| 2 | U2Y2V2 |
| ... | ... |

Figure 18: 4:4:4 format

## 4:2:2 Format

In this format, every two pixels share U and V. This is achieved by using the same U and V values for each pair of pixels. For example, the same U and V values are used for Y1 and Y2, and the same U and V values are used for Y3 and Y4.

| Pixel Number | Pixel Values |
|---|---|
| 0 | 0Y0V0 |
| 1 | U0Y1V0 |
| 2 | U2Y2V2 |
| 3 | U2Y3V2 |
| 4 | U4Y4V4 |
| ... | ... |

Figure 19: 4:2:2 format

In the FOURCC format, the above representation is saved as follows:

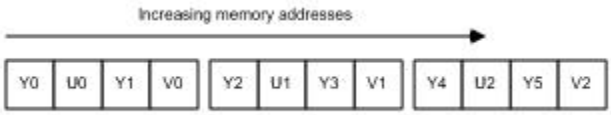



Figure 20: Memory view of 4:2:2

The first four bytes represent two pixels. Both pixels have the same U and V values but different Y values. Note that U and V in the Figure 19 are subsampled entries of the corresponding U and V channels.

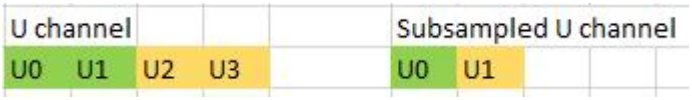



Figure 21: Subsample entries for 4:2:2

In the model, subsampling is done horizontally for each row of the U and V channels, and the YUYV format is implemented.

## Configuration Parameters

| Parameters | Details |
|---|---|
| ***is***_enable | Will enable or disable this module:<br>False: Disable<br>True: Enable |
| ***conv***_type | Set conversion format of $YC_rC_b$ to YUV. Can only be two values:<br>- 444<br>- 422 |

Table 20: YUV Formats configuration parameters

# Pipeline Results

Here are the results of this pipeline compared with a market-competitive ISP. The model outputs are displayed on the right, with the underlying ground truths on the left.

| Ground Truths | Infinite-ISP Reference Model |
| --- | --- |





Figure 22: Pipeline Results

# IQ Metrics Analysis

| Images | PSNR | SSIM |
|---|---|---|
| *Indoor1* | 21.9826 | 0.8891 |
| *Outdoor1* | 17.7066 | 0.8959 |
| *Outdoor2* | 18.4617 | 0.8316 |
| *Outdoor3* | 15.9183 | 0.7351 |
| *Outdoor4* | 22.7322 | 0.8992 |

Table 21: IQ Metrics Analysis

# References


1. Measurement for optoelectronic conversion functions (OECFs) of the digital still-picture camera - NASA/ADS (harvard.edu)
2. https://www.imatest.com/?s=oecf
3. https://www.imatest.com/docs/esfriso_instructions/
4. https://slideplayer.com/slide/5759755/
5. ISO - ISO 14524:2009 - Photography — Electronic still-picture cameras — Methods for measuring optoelectronic conversion functions (OECFs)
6. (PDF) Linearisation of RGB Camera Responses for Quantitative Image Analysis of Visible and UV Photography: A Comparison of Two Techniques (researchgate.net)
7. Linearization in detail - three different ways to linearize images (image-engineering.de)
8. https://patentimages.storage.googleapis.com/f9/11/65/a2b66f52c6dbd4/US8538199.pdf
9. https://static.aminer.org/pdf/PDF/000/319/504/large_scale_infographic_image_downsizing.pdf
10. https://www.ipol.im/pub/art/2011/g_mhcd/article.pdf
11. https://www.ipol.im/pub/art/2011/bcm_nlm/article.pdf
12. https://www.itu.int/rec/R-REC-BT.709-6-201506-I/en
13. https://www.flir.com/support-center/iis/machine-vision/knowledge-base/understanding-yuv-data-formats/
14. https://en.wikipedia.org/wiki/Chroma_subsampling
15. https://www.cs.auckland.ac.nz/courses/compsci773s1c/lectures/YuY2_files/intro.htm#YV12
16. https://learn.microsoft.com/en-us/windows/win32/medfound/recommended-8-bit-yuv-formats-for-video-rendering
17. https://sci-hub.hkvisa.net/10.1109/ICAIIS49377.2020.9194921
18. https://sci-hub.hkvisa.net/10.1109/apccas.2012.6419046
19. https://www.atlantis-press.com/article/25875811.pdf
20. https://www.sciencedirect.com/science/article/abs/pii/0016003280900587
21. https://library.imaging.org/admin/apis/public/api/ist/website/downloadArticle/cic/12/1/art00008
22. https://opg.optica.org/josaa/viewmedia.cfm?uri=josaa-31-5-1049&seq=0
23. https://www.hindawi.com/journals/tswj/2014/979081/
24. https://www.semanticscholar.org/paper/Contrast-Limited-Adaptive-Histogram-Equalization-Zuiderveld/726c95fa3bd47401befc7513b6e52d1c806f26af.
25. https://web.archive.org/web/20120113220509/http://radonc.ucsf.edu/research_group/jpouliot/tutorial/HU/Lesson7.htm
26. https://github.com/cruxopen/openISP
27. https://github.com/QiuJueqin/fast-openISP